

## Paradigmas y Lenguajes de Programación

### Tema Investigado: Haskell

#### Integrantes:

Franco, Mariano Sebastián

Kaczuba, Ivar Tobías

López Carzino, Martina

Meza Arrúa, Francisco

### Fundamentos del Diseño Funcional Adoptado

Para el desarrollo del sistema de control del semáforo lo desarrollamos en el paradigma funcional de Lisp. La ventaja principal de este enfoque es que nos permite modelar el comportamiento del semáforo como una serie de transformaciones matemáticas limpias sobre los datos, en lugar de estar alterando variables en la memoria de la computadora de forma desordenada.

A partir del código implementado, estructuramos y clasificamos nuestros componentes bajo los pilares esenciales de este paradigma:

**Modelado mediante Funciones Puras (Módulo Núcleo):** Nuestras funciones centrales, *transicion* y *timer*, cumplen estrictamente con la definición de funciones puras. Esto significa que no producen efectos secundarios (no alteran nada fuera de ellas mismas) y que, ante los mismos argumentos de entrada, garantizan siempre el mismo resultado de salida.

Por ejemplo, la función *timer* recibe una marca de tiempo (timestamp) y mediante la aritmética mod (módulo), calcula de forma exacta la posición dentro del ciclo de 225 segundos totales. No importa cuántas veces llamemos a esta función con el segundo 150, el resultado siempre va a ser el símbolo 'verde'. Esto hace que el núcleo del sistema sea fácil de probar y totalmente aislado de errores externos.

**Controladores con Efectos Secundarios (Funciones Impuras):** Por el requerimiento, el sistema necesita interactuar con el entorno para dejar registros. Funciones como *registrar-cambio* e *informe* son clasificadas como impuras. Su objetivo principal no es

retornar un valor matemático, sino generar efectos secundarios controlados que son obligatorios para la auditoría, es decir imprimir los estados en tiempo real en la terminal y escribir físicamente el archivo de texto *informe-ejecucion-semaforo.txt*. Al separar la lógica pura (el cálculo del color) de la impura (la escritura de archivos), el diseño del software queda ordenado y compartimentado.

**Inmutabilidad e Impacto No Destructivo en Memoria:** En la programación tradicional o imperativa, cuando un semáforo cambia de color, se suele sobrescribir una variable en la memoria (por ejemplo: `color = "Verde"` borra el valor "Rojo"). En nuestro diseño funcional aplicamos el principio de inmutabilidad. La función transición no destruye el estado anterior, recibe el color-actual y el evento cambiar-a como datos fijos de lectura, evalúa la condición mediante el `cond` y construye una estructura de datos totalmente nueva en la memoria mediante `list` (por ejemplo (`list 'verde "cambiar-a-verde"`)). Los datos originales nunca se corrompen ni se modifican, lo que asegura que el historial del sistema se mantenga funcional.

## **Justificación de las Librerías Seleccionadas (Fase 2)**

En la Fase 2 del proyecto, el sistema requería incorporar un módulo de auditoría capaz de transformar las marcas de tiempo crudas (segundos desde el origen del sistema) en fechas y horas del calendario real que un ser humano pueda entender.

Para resolver este problema de manera eficiente, justificamos la integración de la librería externa *local-time*, la cual cargamos de manera dinámica mediante el gestor de paquetes *Quicklisp* con la instrucción (`ql:quickload :local-time`).

Los motivos de esta elección se basan en las limitaciones propias del lenguaje y las ventajas que aporta esta herramienta las cuales creemos que son:

**Deficiencia del Core de Common Lisp:** El estándar nativo de Common Lisp maneja las funciones de tiempo de una forma muy tosca, lo que nos hubiera obligado a escribir muchas líneas de código extra y realizar cálculos matemáticos manuales muy complejos (dividir segundos por 60, por 24, calcular años bisiestos a mano, etc ) para descomponer un número gigante en una fecha legible.

**Conversión Eficiente con universal-to-timestamp:** Esta función específica de la librería resuelve de forma directa el acoplamiento con el tiempo universal. En nuestro código realiza la matemática necesaria sumando la diferencia de épocas (+ epoch 2208988800) y empaqueta el resultado en un objeto de tiempo estructurado y seguro, reduciendo el margen de error de cálculo.

**Flexibilidad de Formateo mediante format-timestring:** El requerimiento del informe exigía que la fecha tuviera una estructura muy limpia (Año-Mes-Día Hora:Minutos:Segundos). "*format-timestring*" nos permitió resolver el problema de una forma más efectiva (como `(:year #\-) o (:hour #\:))`). Esto nos permitió mapear los

datos internos de la librería directamente al formato de texto deseado en una sola línea de código, garantizando una salida prolija tanto en los logs de la consola como en el archivo de texto exportado.

## **Desarrollo del Lenguaje Asignado: Haskell**

### **Presentación Breve del Lenguaje y Uso en la Industria:**

Haskell es un lenguaje de programación puramente funcional. A diferencia de otros lenguajes donde uno mezcla estilos, Haskell te obliga a programar siguiendo las reglas matemáticas de las funciones de principio a fin. Nació a finales de los 80 con el objetivo de unificar las investigaciones sobre programación funcional en un solo lenguaje moderno.

Hoy en día aunque no es un lenguaje masivo como Python o JavaScript, se usa un montón en industrias que necesitan que sus sistemas sean ultra seguros, estables y donde un error de datos pueda costar mucho dinero. Sus principales áreas de aplicación son:

Finanzas y Banca: Para calcular algoritmos de riesgo y transacciones complejas.

Ciberseguridad y Redes: Para crear sistemas de encriptación que no puedan ser vulnerados.

Sistemas Aeroespaciales y de Defensa: Donde el software no puede fallar bajo ninguna circunstancia.

### **Empresas conocidas que lo utilizan:**

Meta (Facebook): Lo usa para sus sistemas internos de filtrado de spam y herramientas de seguridad.

Barclays / Standard Chartered: Bancos gigantescos que procesan parte de sus operaciones financieras con Haskell.

Microsoft: Lo utiliza en sus laboratorios de investigación y para el desarrollo de herramientas de compilación.

## **Ejes Comparativos Específicos (Haskell)**

1-Common Lisp tiene un tipado dinámico. Haskell tiene un tipado estático estricto. ¿Cómo modelaron los colores del semáforo? Expliquen qué es un Algebraic Data Type (ADT) y cómo el compilador de Haskell impide pasar un estado inválido antes de que el programa se ejecute.

En Common Lisp usamos un tipado dinámico, lo que significa que el lenguaje se entera de qué tipo es un dato recién cuando el programa se está ejecutando. Por eso para los

colores usamos símbolos sueltos ('en-rojo, 'verde). El peligro de esto es que si nos equivocamos al tipear y ponemos 'vverde, el programa arranca igual y falla en el medio de la ejecución.

En Haskell, al tener un tipado estático estricto, el compilador revisa todo ANTES de arrancar el programa. Para modelar los colores del semáforo usamos lo que se conoce como un Algebraic Data Type (ADT) o Tipo de Datos Algebraico, mediante la línea: `data Estado = Rojo | Verde | Amarillo | AmarilloIntermitente`.

Un ADT es básicamente una forma de crear nuestro propio tipo de datos personalizado diciéndole al lenguaje: "la variable Estado solo puede valer una de estas cuatro opciones fijas". La buena de esto es que el compilador de Haskell impide por completo pasar un estado inválido. Si en el código intentamos pasar el color "Azul" o tipeamos mal un nombre, Haskell salta con un error en la pantalla y ni siquiera te deja generar el programa. El error se frena en la fábrica, nunca llega al semáforo real.

## 2-Expliquen cómo afecta la Evaluación Perezosa (Lazy Evaluation) de Haskell al procesamiento de los tiempos del temporizador si tuvieran un flujo infinito de eventos de tráfico.

Haskell maneja una estrategia única llamada Evaluación Perezosa. Esto significa que el lenguaje no calcula el valor de una función o de una operación matemática hasta que no sea estrictamente necesario mostrar el resultado en la pantalla o usarlo para otra cosa.

Si tuviéramos un flujo infinito de eventos de tráfico (por ejemplo sensores de la calle mandando millones de marcas de tiempo por segundo sin parar) en un lenguaje común la computadora se colgaría o se quedaría sin memoria intentando procesar toda la lista extensa.

En Haskell gracias a la evaluación perezosa, podemos tener conceptualmente una lista infinita de tiempos guardada en el sistema. El temporizador (timer) solo va a calcular el color del semáforo para el auto que está cruzando en ese segundo exacto. El resto de la lista infinita del futuro se queda esperando en pausa, sin consumir memoria ni procesador. Esto hace que el procesamiento de flujos masivos de datos sea eficiente y que el sistema nunca se sature.

## **Conclusión del grupo sobre la experiencia:**

La experiencia de estudiar y utilizar un lenguaje del paradigma funcional como Haskell fue un desafío enorme para el grupo. Veníamos acostumbrados a programar de forma imperativa (pensando en variables que cambian todo el tiempo y bucles tradicionales) por lo que cambiar esa forma de pensar todo como ecuaciones matemáticas nos costó bastante al principio.

Haskell nos resultó muy estricto y difícil de hacer andar de entrada (las instalaciones en Windows y los errores de tipos) pero una vez que entendimos cómo funciona su

compilador nos dio más seguridad porque sabes que si compilas desde el inicio el programa no va a fallar durante la ejecución. Aprendimos que Haskell es una herramienta excelente para crear un software ordenado, limpio y muy seguro ante fallos de datos.

## **Errores/Bugs que tuvimos y como los resolvimos:**

### **Bug 1:**

Error de “función indefinida” (EVAL: la función MI-FUNCION no está definida)

Al querer ejecutar el comando PROBAR-TODO, se frenó por completo y tiró el error  
\*\*\* - EVAL: la función MI-FUNCION no está definida.

Después de revisar un poco nos dimos cuenta de que este error saltó porque estábamos queriendo usar un nombre genérico de prueba (MI-FUNCION) que directamente no existía en nuestro archivo. En Lisp si llamas a algo que no guardaste antes, el entorno obviamente no encuentra que ejecutar.

Para solucionarlo nos guiamos por el código final de nuestro archivo. Reemplazamos esas llamadas genéricas por el nombre real ejecutar-suite-pruebas. Al cargar el archivo final completo y escribir (ejecutar-suite-pruebas), el sistema reconoció la función y corrió todas las pruebas de transición, el timer y los reportes sin fallar.

```
PROBAR-TODO
[8]>
*** - EVAL: la función MI-FUNCION no está definida
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of (FDEFINITION 'MI-FUNCION).
RETRY          :R2      Reintentar
STORE-VALUE    :R3      Input a new value for (FDEFINITION 'MI-FUNCION).
ABORT          :R4      Abort main loop
Break 1 [0]
```

### **Bug 2:**

Al momento de cargar la librería externa local-time e intentar procesar las funciones del semáforo para solucionar el cálculo de los años Lisp interrumpió la compilación mostrando este mensaje: \*\*\* - SYSTEM::LINE-COMMENT-READER: Invalid byte #x81 in CHARSET:CP1252 conversion

Al procesar las marcas de tiempo Epoch con la función universal-to-timestamp de la librería local-time, las fechas daban cualquier cosa (se iban al año 2096 o se reseteaban). Esto pasaba porque Common Lisp computa su tiempo nativo desde el 1

de enero de 1900, mientras que el estándar UNIX (Epoch) inicia en 1970. Había que meter una suma matemática en el código para corregir esa diferencia de épocas.

Para solucionar esto primero limpiamos el archivo de texto borrando todos los acentos raros de los comentarios para que el formato CP1252 de la consola no tire más el error de lectura. Una vez que el código volvió a compilar limpio, sumamos la diferencia exacta de segundos (+ epoch 2208988800) dentro de la función. Con eso el sistema empezó a mostrar los años correspondientes de forma correcta.

```
To load "local-time":
  Load 2 ASDF systems:
    asdf uiop
  Install 1 Quicklisp release:
    local-time
; Fetching #<URL "http://beta.quicklisp.org/archive/local-time/2026-01-01/local-time-20260101-git.tgz">
; 550.30KB
=====
563,503 bytes in 0.31 seconds (1761.01KB/sec)
; Loading "local-time"
[package local-time]...
(:LOCAL-TIME)
[5]>
TRANSICION
[6]>
TIMER
[7]>
REGISTRAR-CAMBIO
[8]>
*** - SYSTEM::LINE-COMMENT-READER: Invalid byte #x81 in CHARSET:CP1252 conversion
```

### **Bug 3:**

Al ejecutar el código de Haskell se imprimió el resultado: Tiempo [1901-12-13T20:45:52Z]: la luz cambio de Rojo a AmarilloIntermitente.

Después de investigar bastante, aprendimos que el tipo de datos Int32 (32 bits) tiene un límite numérico máximo. Al sumarle un segundo más a eso, el espacio de memoria se desbordó y el número se volvió negativo. En vez de avanzar hacia el año 2038, el reloj interno del sistema empezó a contar hacia atrás y por eso llego al año 1901.

Para solucionar esto modificamos el código para cambiar el tipo de datos de Int32 a Int64 (64 bits). Al agrandar el tamaño de la variable, el sistema ganó espacio para procesar números grandes logrando mostrar la fecha real del futuro (2038-01-19) de forma correcta.

```
=== PRUEBA DE ASEGURAMIENTO DE LA CALIDAD (HASKELL) ===  
Tiempo [1901-12-13T20:45:52Z]: la luz cambio de Rojo a AmarilloIntermitente  
[Finished in 6.2s]
```

## **Explicación de Requerimiento 7:**

El requerimiento 7 exigía garantizar que todas las funciones individuales del semáforo anduvieran bien al juntarse. Para no tener que probar cada función a mano una por una, hicimos una función integradora llamada ejecutar-suite-pruebas.

Esta función es clasificada como impura porque su único objetivo es usar la instrucción `format t` para imprimir de forma interactiva el sistema, además de generar archivos físicos.

También queríamos explicar que hace cada bloque de este requerimiento:

Control de Transiciones Básicas: Las primeras líneas ponen a prueba la lógica de la función `transicion`. Le pasamos tres escenarios diferentes: un caso normal (pasar de rojo a verde que debería dar el paso intermedio), una transición alternativa (de intermitente a verde), y un caso inválido a propósito (querer pasar de rojo a amarillo peligrosas o incorrectas).

Validación del Temporizador (timer): El segundo bloque le manda marcas de tiempo fijas (0, 91 y 150) a la función del reloj. Con esto nos aseguramos de que matemáticamente los tiempos caigan exactamente en los colores que corresponden según los rangos que definimos en la consigna (rojo, intermitente y verde respectivamente).

Simulación de Auditoría y Tiempos Extremadamente Largos: En la mitad de la suite llamamos a `registrar-cambio` pasándole una marca de tiempo real en formato UNIX (1774881600). Esto sirve para probar que la librería `local-time` esté convirtiendo los segundos a una fecha legible de forma correcta. Después abajo usamos un bloque `let` para calcular la duración total del ciclo (225 segundos) y ver si el sistema nos devuelve la recomendación de diseño psicológico correcta para los conductores.

Cálculos de Planificación y Estadísticas: Las siguientes líneas prueban las funciones matemáticas del semáforo. Le pedimos que calcule cuántos ciclos completos daría el semáforo en 15 minutos y en una hora, y que nos devuelva la lista con la distribución porcentual del tiempo de cada luz.

*Prueba de Persistencia en Disco:* Al final de todo, la función le manda datos simulados a informe. Esto lo hacemos para comprobar que el programa sea capaz de abrir, escribir y cerrar correctamente el archivo de texto externo (informe-ejecucion-semaforo.txt) en el disco de la computadora, avisándonos con un mensaje de éxito en la pantalla si todo salió bien.

### **Bibliografía:**

Graham, P. (1995). *ANSI Common Lisp*. Prentice Hall. Enlace a recursos oficiales de Common Lisp <https://common-lisp.net/>

Local-Time Team. (2024). *The Local-Time Manual*. Registro oficial del paquete en Quicklisp <https://quickdocs.org/local-time>

Haskell Org. (2026). *Haskell Language Official Documentation*. Documentacion central del lenguaje <https://www.haskell.org/documentation/>

Marlow, S. (2010). *Haskell 2010 Language Report*. Estándar oficial del lenguaje y definicion de tipos de datos básicos <https://www.haskell.org/onlinereport/haskell2010/>